

```
fun length([]) = 0
  | length(a::x) = 1 + length(x)
```

Tímto zápisem vlastně popisujeme, že pro prázdný seznam je jeho délka rovna nule a pro seznam skládající se z hlavy a těla je délka tohoto seznamu rovna délce těla + 1 (což je elegantní rekurzivní vyjádření nějaké vlastnosti, s níž se velmi často setkáme v matematice).

Podobným způsobem lze zapsat výpočet Fibonacciho řady, resp. přesněji řečeno n -tého prvku této řady, tentokrát se třemi větvemi:

```
fun fib 0 = 0
  | fib 1 = 1
  | fib n = fib (n - 1) + fib (n - 2);
```

```
fib 0;
fib 1;
fib 10;
```

Poznámka: podle mého názoru se jedná o zcela nejpříjemnější způsob zápisu, jaký je možný, čitelný pro člověka a zpracovatelný počítačem.

Strukturální pattern matching v programovacím jazyku F#

Prakticky stejným způsobem lze zapsat výpočet v jazyku F#, který je, jak již ostatně víme, potomkem jazyka ML:

```
let rec fib n =
  match n with
  | 0 -> 0
  | 1 -> 1
  | n -> fib(n-1) + fib(n-2)
```

```
printfn "Fib 0=%i" (fib 0)
printfn "Fib 1=%i" (fib 1)
printfn "Fib 10=%i" (fib 10)
```

Poznámka: až na nepatrné syntaktické odlišnosti (a jednu odlišnost v sémantice – explicitní vyjádření rekurzivní funkce), je tento zápis totožný s předchozím zápisem, takže nevyžaduje podrobnější popis.

Strukturální pattern matching v Rustu

Programovací jazyk *Rust* sice nepodporuje (ostatně podobně jako Python) klasickou céčkovskou rozhodovací konstrukci typu `switch-case` (kterou převzala a jen mírně rozšířila Java atd.), což však ve skutečnosti není žádná škoda, protože namísto toho nabízí programátorům mnohem robustnější konstrukci založenou na klíčovém slově `match`, kterou lze taktéž použít pro volbu některé větve kódu, která se má při splnění podmínky resp. při nalezení vzoru provést. Nejprve se bez dlouhých popisů podívejme na způsob zápisu této konstrukce v její nejjednodušší podobě, tedy při klasickém rozvětvení na základě aktuální hodnoty proměnné `x`:

```
fn main() {
    let x:i32 = 1;

    match x {
        0 => println!("zero"),
        1 => println!("one"),
        2 => println!("two"),
        3 => println!("three"),
        _ => println!("something else"),
    }
}
```

Z tohoto demonstračního příkladu je patrné, že v konstrukci `match` je zapsáno několik větví, přičemž každá z těchto větví začíná porovnáváním hodnotou a výraz či příkaz je uveden za šipkou tvořenou znaky `=>`. Poslední řádek obsahuje symbol `_`, který vlastně nahrazuje větev `default`. Ve skutečnosti je však sémantika tohoto symbolu poněkud odlišná, protože před šipkou `=>` je uveden takzvaný vzorek (*pattern*) a nikoli pouhopouhá konstanta, což si ostatně ukážeme na dalších příkladech.

V době překladu zdrojového kódu překladač Rustu kontroluje, zda je symbol `_` použit, protože je vyžadováno, aby větve v konstrukci `match` pokryly všechny možné hodnoty testovaného výrazu. Zkusme si tedy pro ověření tohoto tvrzení přeložit následující příklad:

```
fn main() {
    let x:i32 = 1;
```

```
match x {  
    0 => println!("zero"),  
    1 => println!("one"),  
    2 => println!("two"),  
    3 => println!("three"),  
}
```

Podle očekávání skončí pokus o překlad tohoto zdrojového kódu s chybou.

Podobně, jako tomu bylo u programové konstrukce `if-else` či u programového bloku `{}`, je i konstrukce `match` považována za *výraz*, nikoli za *příkaz*. To znamená, že celou konstrukci `match` je možné použít v nějakém výpočtu, uvnitř jiného výrazu atd. V dalším demonstračním příkladu je deklarována funkce nazvaná `classify`, které se předá celočíselná hodnota a funkce vrátí konstantní řetězec, který tuto hodnotu popisuje. Tato funkce obsahuje ve svém bloku jediný výraz a tím je `match` (proto ostatně za složenou závorkou není středník):

```
fn classify(x:i32) -> &'static str {  
    match x {  
        0 => "zero",  
        1 => "one",  
        2 => "two",  
        3 => "three",  
        _ => "something else",  
    }  
}  
  
fn main() {  
    for x in 0..10 {  
        println!("{}", x, classify(x))  
    }  
}
```

n -tý prvek Fibonacciho řady lze vypočítat rekurzivně, taktéž s využitím pattern matchingu:

```
fn fib(n: u32) -> u32 {  
    match n {  
        0 | 1 => 1,  
        _ => fib(n - 1) + fib(n - 2),  
    }  
}
```

```
fn main() {
  for x in 0..10 {
    println!("{}", x, fib(x))
  }
}
```

Poslední příklad ukazuje rozsahu, kdy se rozpoznají hodnoty 13 až 19:

```
match number {
  1 => println!("One!"),
  2 | 3 | 5 | 7 | 11 => println!("This is a prime"),
  13..=19 => println!("A teen"),
  _ => println!("Ain't special"),
}
```

Poznámka: ve skutečnosti jsou podporovány i mnohem složitější konstrukce pattern matchingu, ovšem nyní již nastává čas věnovat se programovacímu jazyku Python. Uvidíme, že konstrukce pattern matchingu v Pythonu se do značné míry podobají právě Rustu, což ovšem není náhoda, protože právě tudy vedla cesta k moderním formám pattern matchingu. Ostatně právě proto jsme si tyto příklady uváděli.

4.6.2 Strukturální pattern matching v programovacím jazyku Python

V té nejjednodušší variantě se můžeme na strukturální pattern matching dívat jako na poněkud odlišněji zapsanou variantu programové konstrukce `switch`, která doposud v Pythonu chyběla (a to poměrně z dobře diskutovaných důvodů). Ovšem již na následujícím demonstračním příkladu je patrné, že možnosti mohou být poněkud větší (a to se jedná o triviální příklad použití):

```
# Strukturální pattern matching:
# - Výpočet Fibonacciho posloupnost realizovaný s využitím
#   pattern matchingu

def fib(value):
    """Výpočet jednoho prvku Fibonacciho posloupnosti."""
    match value:
        case 0:
            return 0
        case 1:
            return 1
```

— 4 Postupné rozšiřování syntaxe a sémantiky Pythonu

```
case n if n>1:
    return fib(n-1) + fib(n-2)
case _ as wrong:
    raise ValueError("Wrong input", wrong)

# tisk tabulky s prvky Fibonacciho posloupnosti
for n in range(0, 11):
    print(n, fib(n))

# test neplatného vstupu
fib(-1)
```

Zdrojový kód tohoto příkladu můžete najít na adrese [https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/pattern-matching/pattern-matching-fib.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/pattern-matching/pattern-matching-fib.py)

Ve vzorku, tj. ve výrazu zapsaném přímo za klíčovým slovem `case` můžeme v případě potřeby testovat i hodnoty více proměnných či parametrů. Ukažme si to na příkladu rekursivního výpočtu Ackermannovy funkce (s ní se v této knize setkáme několikrát, je to totiž vhodný „školní“ příklad). Zde postupně zkoumáme, jestli je první parametr m nulový, ve druhé větvi testujeme, jestli je naopak druhý parametr n nulový a ve větvi třetí již máme jistotu, že nulový *není* ani jeden z těchto parametrů.

Realizace výpočtu Ackermannovy funkce bez použití pattern matchingu by mohla vypadat následovně:

```
# Výpočet Ackermannovy funkce, založeno na konstrukci if

def A(m, n):
    """Ackermannova funkce."""
    if m == 0:
        return n + 1
    if n == 0:
        return A(m - 1, 1)
    return A(m - 1, A(m, n - 1))

# otestování korektnosti výpočtu Ackermannovy funkce
for m in range(4):
    for n in range(5):
        print(m, n, A(m, n))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/pattern-matching/ackermann-if.py

Popř. můžeme použít skutečné rozvětvení:

```
# Výpočet Ackermannovy funkce, založeno na konstrukci if-elif-else
```

```
def A(m, n):  
    """Ackermannova funkce."""  
    if m == 0:  
        return n + 1  
    elif n == 0:  
        return A(m - 1, 1)  
    else:  
        return A(m - 1, A(m, n - 1))
```

```
# otestování korektnosti výpočtu Ackermannovy funkce  
for m in range(4):  
    for n in range(5):  
        print(m, n, A(m, n))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/pattern-matching/ackermann-if-else.py

Od tohoto stylu zápisu je již přechod k bloku `match` relativně přímočarý a po krátkém zaučení nejenom čitelnější, ale navíc můžeme mít jistotu, že jsou prozkoumány všechny možné případy:

```
# Strukturální pattern matching:  
# - Výpočet Ackermannovy funkce
```

```
def A(m, n):  
    """Ackermannova funkce."""  
    match (m, n):  
        case (0, n):  
            return n + 1  
        case (m, 0):  
            return A(m-1, 1)  
        case (m, n):  
            return A(m - 1, A(m, n - 1))
```

```
# otestování korektnosti výpočtu Ackermannovy funkce
for m in range(4):
    for n in range(5):
        print(m, n, A(m, n))
```

Zdrojový kód tohoto příkladu můžete najít na adrese https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-ackermann.py

V dalším textu si uvedeme i poněkud komplikovanější příklady použití pattern matchingu.

4.6.3 Rozvětvení realizované konstrukcí `match`

V té nejjednodušší podobě může konstrukce `match` nahradit rozvětvení s větším množstvím větví, které je v programovacím jazyku Python typicky realizováno sekvencí bloků `if-elif-else`. Podívejme se na jednoduchý příklad, v němž je toto rozvětvení použito pro simulaci odpovědi na otázku, na kterou si možná někteří pamětníci vzpomenu z dob kralování DOSu. Operační systém DOS totiž při problémech se čtením či zápisem dat na paměťové médium zobrazil otázku „Abort, Retry, Fail?“ a očekával přitom jednoznakovou odpověď, tedy zápis znaku „a“, „r“ nebo „f“. Jakákoli jiná odpověď byla považována za chybnou.

Vyzkoušejme si první variantu, jak tento dotaz a reakci na odpověď realizovat:

```
# Strukturální pattern matching:
# - použití rozhodovací konstrukce if-elif-else
#   namísto pattern matchingu

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    if response == "a":
        return "Abort"
    elif response == "r":
        return "Retry"
    elif response == "f":
        return "Fail"
```

— 4 Postupné rozšiřování syntaxe a sémantiky Pythonu

```
    else:
        return "Wrong response"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-1.py

Jedna z nabízených možností náhrady sekvence bloků `if-elif-else` je založena na použití mapy (slovníku), která v klíčích obsahuje očekávanou odpověď a hodnoty by měly obsahovat prováděnou operaci (či plnou odpověď). Jedná se o poměrně elegantní řešení, které bylo v Pythonu dlouhou dobu preferováno:

```
# Strukturální pattern matching:
# - rozhodování realizované slovníkem (mapou)
#   namísto pattern matchingu

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # odpovědi a odpovídající operace
    commands = {
        "a": "Abort",
        "r": "Retry",
        "f": "Fail",
    }

    # rozhodnutí o provedené operaci na základě odpovědi
    return commands.get(response, "Wrong response")

print(abort_retry_fail())
```


Zdrojový kód tohoto příkladu můžete najít na adrese https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-2.py

Nás ovšem bude v této kapitole zajímat především nová konstrukce `match`. Tuto konstrukci lze použít následovně:

```
# Strukturální pattern matching:
# - rozhodování realizované pattern matchingem

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    match response:
        case "a":
            return "Abort"
        case "r":
            return "Retry"
        case "f":
            return "Fail"
        case _:
            return "Wrong response"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-3.py

Povšimněte si zejména posledního bloku `case`, v němž se opět používá speciální znak `_`. Tento znak, pokud je použit ve vzoru, bude odpovídat jakémukoli vstupu.

Programovací jazyk Python, na rozdíl od výše zmíněného Rustu (nebo jazyků odvozených od ML), však nevyžaduje, aby byly pokryty všechny možné varianty, které mohou nastat. To znamená, že předchozí program lze zkrátit na (vynechat poslední větev):

— 4 Postupné rozšiřování syntaxe a sémantiky Pythonu

```
# Strukturální pattern matching:
# - rozhodování realizované pattern matchingem

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    match response:
        case "a":
            return "Abort"
        case "r":
            return "Retry"
        case "f":
            return "Fail"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-4.py

V případě neočekávaného vstupu se v tomto případě vrátí hodnota (nebo raději symbol představující neexistující hodnotu) `None`. To ovšem není specifické chování bloku `match`, ale obecné chování Pythonu – na funkci nevracející žádnou hodnotu se můžeme dívat i jako na funkci, která ve skutečnosti vrací hodnotu `None`. To si mimochodem ověříme:

```
def x():
    pass

y = x()
print(y)
```

Měla by se skutečně vypsát hodnota `None`.

4.6.4 Vzory obsahující v každé větvi větší množství hodnot

V dalším kroku se pokusme předchozí demonstrační příklady rozšířit takovým způsobem, aby skript reagoval nejenom na jednoznakové odpovědi „a“, „r“ a „f“, ale například i na zápis odpovědi velkými písmeny, na zápis celých slov a podobně. Ve „starém“ Pythonu spočívá jedno možné řešení ve využití množinové operace `in` použité ve větvích `if` a `elif`. Řešení založené na použití množin tedy může vypadat následovně:

```
# Strukturální pattern matching:
# - použití rozhodovací konstrukce if-elif-else
# - podpora odpovědí psaných velkými i malými znaky

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    if response in {"a", "A", "abort", "Abort", "ABORT"}:
        return "Abort"
    elif response in {"r", "R"}:
        return "Retry"
    elif response in {"f", "F"}:
        return "Fail"
    else:
        return "Wrong response"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-5.py

V případě využití nové programové konstrukce `match` je možné v každém vzoru (a tedy ve výrazu zapsaném za slovem `case`) použít operátor „or“, který je zapisovaný znakem `|`. Zápis je tak relativně stručný, což je ostatně patrné i z následující ukázky, která provádí podobnou činnost, jako skript uvedený v předchozím odstavci:

```
# Strukturální pattern matching:
# - rozhodování realizované pattern matchingem
# - podpora odpovědí psaných velkými i malými znaky

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    match response:
        case "a" | "A" | "abort" | "Abort" | "ABORT":
            return "Abort"
        case "r" | "R":
            return "Retry"
        case "f" | "F":
            return "Fail"
        case _:
            return "Wrong response"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-6.py

Poznámka: povšimněte si, jak je tento zápis podobný zápisu pattern matchingu v programovacím jazyku Rust. Opět se nejedná o náhodu.

4.6.5 Zachycení hodnoty proměnné v rozhodovací větvi

Následující demonstrační příklad je prozatím poněkud umělý. Je v něm ukázáno, jak lze v zápisu vzorku ve větvi `case` zachytit hodnotu (nebo hodnoty). Jedná se o poslední větev typu „else“ či „default“, v níž libovolnou vstupní hodnotu zachytíme do lokální proměnné nazvané `x`, kterou dále v této větvi použijeme při zobrazení chybového hlášení:

```
# Strukturální pattern matching:
# - rozhodování realizované pattern matchingem
# - podpora odpovědí psaných velkými i malými znaky
# - zachycení neočekávané odpovědi

print("Not ready reading drive A")

def abort_retry_fail():
    # získání odpovědi od uživatele
    response = input("Abort, Retry, Fail? ")

    # rozhodnutí o provedené operaci na základě odpovědi
    match response:
        case "a" | "A":
            return "Abort"
        case "r" | "R":
            return "Retry"
        case "f" | "F":
            return "Fail"
        case _ as x:
            return f"Wrong response {x}"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-7.py

Výše uvedený demonstrační příklad je umělý z toho důvodu, že v tomto konkrétním případě můžeme přímo použít proměnnou `response`, která je pochopitelně uvnitř celého bloku `match` viditelná a tím pádem ji můžeme číst. Ovšem po nepatrné úpravě zdrojového kódu můžeme dojít do situace, ve které žádná taková proměnná nebude existovat a tudíž bude skutečně nutné použít zápis s klíčovým slovem `as`:

```
# Strukturální pattern matching:
# - rozhodování realizované pattern matchingem

print("Not ready reading drive A")
```

```
def abort_retry_fail():
    # získání odpovědi od uživatele
    # a rozhodnutí o provedené operaci
    match input("Abort, Retry, Fail? "):
        case "a" | "A":
            return "Abort"
        case "r" | "R":
            return "Retry"
        case "f" | "F":
            return "Fail"
        case _ as x:
            return f"Wrong response {x}"

print(abort_retry_fail())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/pattern-matching/pattern-matching-abort-retry-fail-8.py

4.6.6 Podmínka zapsaná v rozhodovacích větvích konstrukce `match`

Vyzkoušejme si nyní zapsat rekurzivní výpočet faktoriálu s využitím `pattern matchingu`. První varianta realizace takového výpočtu může vypadat následovně:

```
# Strukturální pattern matching:
# - výpočet faktoriálu s využitím pattern matchingu
# - základní varianta akceptující neplatné vstupy

def factorial(n):
    """Rekurzivní výpočet faktoriálu."""
    match n:
        case 0:
            return 1
        case 1:
            return 1
        case x:
            return x * factorial(x-1)

# tisk tabulky faktoriálů
for i in range(0, 10):
    print(i, factorial(i))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/pattern-matching/pattern-matching-factorial1.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/pattern-matching/pattern-matching-factorial1.py)

Až na nepatrně odlišnou rekurzi se jedná o prakticky shodný programový kód, jako tomu bylo v příkladu s prvky Fibonacciho posloupnosti. Navíc si povšimněte, že vzorek x odpovídá libovolné hodnotě, přičemž hodnoty 0 a 1 se zde již nebudou vyskytovat, protože ty byly zachyceny v předchozích větvích.

Ve skutečnosti lze ovšem snadno dokázat, že takto realizovaný výpočet faktoriálu nebude korektní pro záporné vstupy. Z tohoto důvodu do řídicí struktury `match` dopíšeme podmínku omezující hodnotu zachycené proměnné x . To nám programovací jazyk Python v konstrukci `match` umožňuje:

```
# Strukturální pattern matching:
# - výpočet faktoriálu s využitím pattern matchingu
# - test, zda je vstup nezáporný

def factorial(n):
    """Rekurzivní výpočet faktoriálu."""
    match n:
        case 0:
            return 1
        case 1:
            return 1
        case x if x>1:
            return x * factorial(x-1)
        case _:
            raise TypeError("expecting integer >= 0")

# tisk tabulky faktoriálů
for i in range(-1, 10):
    try:
        print(i, factorial(i))
    except Exception as e:
        print(e)
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/pattern-matching/pattern-matching-factorial2.py

Pro záporné hodnoty by měl výpočet vyhazovat výjimku `TypeError`, kterou zachytíme a vypíšeme namísto ní vhodné chybové hlášení:

```
expecting integer >= 0
0 1
1 1
2 2
3 6
4 24
5 120
6 720
7 5040
8 40320
9 362880
```

Můžeme dokonce deklarovat (prozatím ovšem nepříliš elegantně), jakého typu má hodnota být. Později si ukážeme mnohem vylepšené řešení:

```
# Strukturální pattern matching:
# - výpočet faktoriálu s využitím pattern matchingu
# - test, zda má vstup korektní typ

def factorial(n):
    """Rekurzivní výpočet faktoriálu."""
    match n:
        case 0:
            return 1
        case 1:
            return 1
        case x if isinstance(x, int) and x>1:
            return x * factorial(x-1)
        case _:
            raise TypeError("expecting integer >= 0")

# tisk tabulky faktoriálů
for i in range(-1, 10):
    try:
        print(i, factorial(i))
    except Exception as e:
        print(e)
```